C PROGRAMMING

Dynamic Memory Allocation

and Memory Functions

Run-Time Environment | malloc() | free()

Complete Exam-Oriented Study & Quick Revision Guide

STACK

Local variables

Function calls

Grows downward

HEAP

malloc() / free()

Dynamic memory

Grows upward

Topics: Run-Time Environment | Dynamic Memory Management | malloc() | free() | Programs

Table of Contents

1. Concept of Dynamic Environment as Run-Time Environment
2. Concept of Dynamic Memory Management
3. Built-in Dynamic Memory Management Tools of C — malloc() and free()
4. Simple Programs Using malloc() and free()
5. Quick Revision (One Page)

> **How to Use This Document**
> Gold-bordered cream boxes highlight **Key Ideas and definitions**; rose-bordered boxes flag **common mistakes / exam alerts**; blue boxes show exact C **syntax**; dark boxes contain runnable `C code`.

1. Concept of Dynamic Environment as Run-Time Environment

1.1 Meaning of Run-Time Environment

The **run-time environment** of a program is the state of memory and system resources while the program is actually executing. A **dynamic environment** means that the exact amount of memory needed can only be decided **during execution** — for example, based on a value entered by the user — and not fixed in advance while writing the code.

> **Key Idea**
>
> **Run-Time Environment:** The environment (memory layout, Stack, Heap, and system resources) that exists while a program is actively running, allowing memory to be requested and released dynamically.

1.2 Static vs Dynamic Memory Allocation

Because the compiler cannot always know in advance how much memory a program will need, C allows memory to be reserved in two different ways:

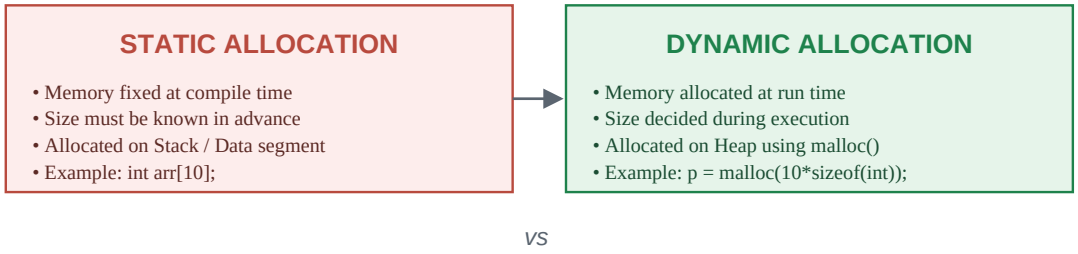

Fig 1.1 — Static allocation (decided at compile time) vs Dynamic allocation (decided at run time)

Basis	Static Allocation	Dynamic Allocation
When decided	Compile time	Run time
Memory region	Stack / Data segment	Heap
Size	Fixed, must be known in advance	Decided during execution
Example	int arr[10];	int *arr = malloc(10*sizeof(int));

1.3 Run-Time Memory Layout

When a C program executes, its memory is organized into segments. The **Stack** holds local variables and function-call information (allocated automatically). The **Heap** is the region used for dynamic memory requested with malloc() and released with free().

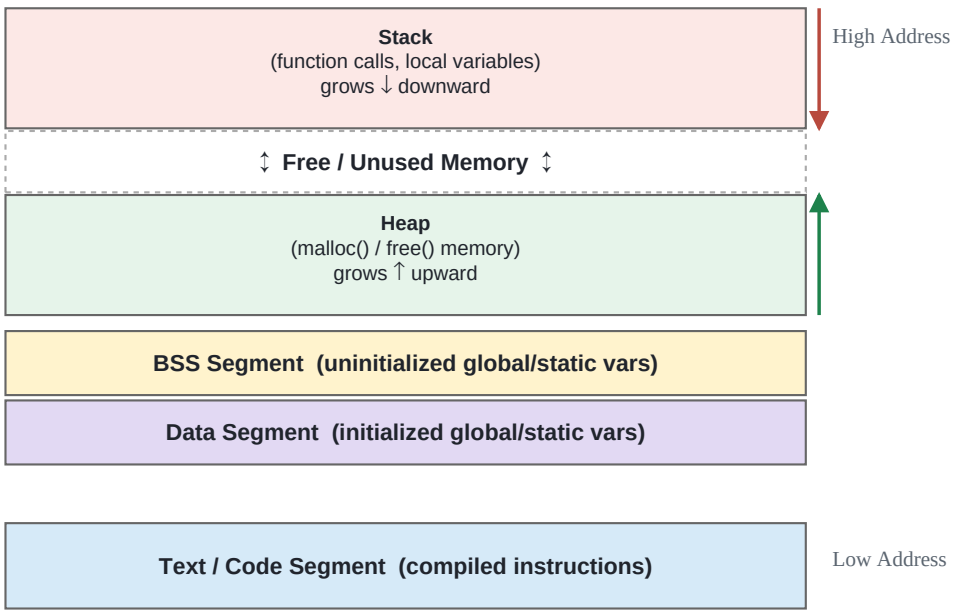

Fig 1.2 — Run-time memory layout of a C program

2. Concept of Dynamic Memory Management

2.1 Definition, Purpose and Need

> **Key Idea**
> **Dynamic Memory Management** is the process of allocating and releasing memory for a program **while it is executing**, using library functions, so that memory usage adapts to the program's actual needs instead of a fixed compile-time guess.

Advantages of Dynamic Memory Management:

- **Efficient use of memory:** reserved only when needed, released when no longer required.
- **Flexibility:** size of arrays / data can be decided from run-time input.
- **Avoids wastage:** no need to over-allocate for a "worst case" that may never occur.
- **Lifetime control:** memory persists until explicitly freed, even after a function returns.

2.2 How Allocation and Release Work (Heap Concept)

The **Heap** is the region of memory used for dynamic allocation. Unlike the Stack, it is **managed manually by the programmer**: a function such as malloc() searches the Heap for a free block of the requested size and returns its address; free() later returns that block to the Heap so it can be reused.

S.No.	Stack	Heap
1.	Allocated and managed automatically by the compiler	Allocated and managed manually by the programmer, using malloc() and free()
2.	Memory is allocated at compile time	Memory is allocated at run time

S.No.	Stack	Heap
3.	Memory is released automatically as soon as the function returns	Memory persists until it is explicitly released using free()
4.	Risk: Stack overflow (rare, e.g. deep recursion)	Risk: Memory leak or dangling pointer if mismanaged

Exam Point

Memory in the Heap is **not** automatically freed when a function returns. The programmer must explicitly release it with free(), otherwise it causes a **memory leak**.

3. Built-in Dynamic Memory Management Tools of C

The C Standard Library provides two core functions, declared in `<stdlib.h>`, for dynamic memory management:

```
#include <stdlib.h>   /* required for malloc() and free() */
```

3.1 malloc() — Memory Allocation

Definition

`malloc()` ("memory allocation") reserves a block of **uninitialized** memory of the requested size on the Heap, and returns a pointer to its first byte — or NULL if allocation fails.

```
void *malloc(size_t size);
```

Element	Description
size	Number of bytes to allocate (usually n * sizeof(type))
Return value	void* to the allocated block, or NULL on failure
Type casting	(int *) malloc(...) — casts void* to the needed pointer type

EXAMPLE: ALLOCATION WITH NULL CHECK

```
int *p = (int *) malloc(5 * sizeof(int));   /* space for 5 integers */

if (p == NULL) {                    /* always check for failure */
    printf("Memory allocation failed!\n");
    exit(1);
}
```

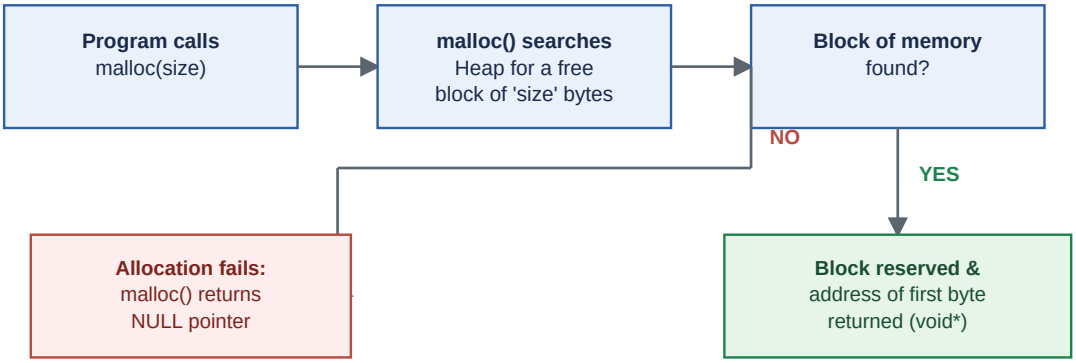

Fig 3.1 — malloc() allocation flow

Fig 3.2 — Pointer p before allocation (garbage) and after malloc() (points to a valid Heap block)

3.2 free() — Memory Deallocation

> **Definition**
> `free()` releases a block of memory previously allocated by malloc(), returning it to the Heap so it can be reused. It returns nothing and does not itself change the pointer variable.

```
void free(void *ptr);
```

After free(p), the block is available for reuse, but p itself still holds the old address — it becomes a **dangling pointer**. Reading/writing through it is undefined behaviour, so good practice is: `free(p); p = NULL;`

Fig 3.3 — free() deallocation flow

Fig 3.4 — Memory state before and after free(p)

Common Mistake	Consequence
Freeing a pointer twice (double free)	Undefined behaviour; may crash / corrupt the Heap
Using a pointer after free() (dangling pointer)	Undefined behaviour; unpredictable output
Forgetting free() entirely	Memory leak
Freeing a pointer not returned by malloc()	Undefined behaviour

Exam Point

Every successful malloc() should be matched with exactly **one** free(). Not freeing causes a memory leak; freeing twice causes undefined behaviour.

4. Simple Programs Using malloc() and free()

The two programs below cover the full malloc()/free() cycle — allocate memory, store and access values through it, and release it — first for an array, then for a single variable.

4.1 Program 1 — Dynamic Array (Sum of N Numbers)

Problem Statement: Allocate memory dynamically for n integers entered by the user, store them, compute their sum, and free the memory.

sum_dynamic_array.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr, n, i, sum = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    arr = (int *) malloc(n * sizeof(int));   /* allocate */
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter %d numbers:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);              /* store */

    for (i = 0; i < n; i++)
        sum += arr[i];                     /* access / use */

    printf("Sum = %d\n", sum);

    free(arr);                             /* release */
    arr = NULL;
    return 0;
}
```

- n is read at run time — exactly why dynamic allocation is needed.
- malloc(n * sizeof(int)) reserves space for n integers; the NULL check guards against failure.
- One loop stores the values entered by the user; a second loop uses them to compute the sum.
- free(arr) releases the block once it is no longer needed.

Expected Output (n = 4, numbers: 5 10 15 20):

OUTPUT

```
Enter number of elements: 4
Enter 4 numbers:
5 10 15 20
Sum = 50
```

4.2 Program 2 — Dynamic Memory for a Single Variable

Problem Statement: Use malloc() to allocate memory for one integer (not an array), store a value in it, display it, and free the memory.

single_value_malloc.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = (int *) malloc(sizeof(int));   /* allocate space for ONE integer */

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    *p = 100;                          /* store a value via the pointer */

    printf("Value stored = %d\n", *p);        /* access */
    printf("Address = %p\n", (void *)p);

    free(p);                           /* release */
    p = NULL;
    return 0;
}
```

- malloc(sizeof(int)) requests exactly enough memory for one integer — malloc() need not always allocate an array.
- *p = 100; stores a value directly by dereferencing the pointer.
- free(p) releases the single allocated block.

Expected Output (address will vary on each run):

OUTPUT (sample)

```
Value stored = 100
Address = 0x55d2f3a1a2b0
```

5. Quick Revision (One Page)

DYNAMIC MEMORY ALLOCATION — AT A GLANCE

Run-Time Environment: memory state during execution; lets a program request/release memory while it runs.

Static vs Dynamic: Static = fixed size at compile time (Stack/Data). Dynamic = size decided at run time (Heap), via malloc()/free().

malloc(): void *malloc(size_t size); → allocates 'size' bytes, returns a pointer (or NULL on failure). Content is uninitialized.

free(): void free(void *ptr); → releases memory back to the Heap. Returns nothing. Always set the pointer to NULL after freeing.

Golden Rule: Every successful malloc() must be matched with exactly one free(). Always check malloc()'s return value for NULL before use.

Avoid: memory leaks (forgetting free), dangling pointers (using memory after free), double free (freeing the same pointer twice).

End of study material.